Extracting Data from PDFs

Week 9 · Documents module · Textbook Chapter 9

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

October 12, 14 & 16, 2026

This week at a glance

PDFs are the web's filing cabinets. This week we pry official records out of them and turn locked documents into data.

Monday | Concepts & notebook intro

- Why PDFs resist extraction; reconstruction vs. reading
- Four steps: find the data, ask for it, parse a born-digital PDF, OCR a scan — by hand first
- In class: the city's revenue report, by hand (20 minutes)

Wednesday | Notebook lab

- Steps 3 and 4 in Python: pypdf on the council minutes, pdfplumber on the revenue reports, OCR on a picture of a table

Friday | Textbook revisions

- Propose & peer-review pull requests improving Chapter 9

Course overview

Module	Week	Dates	Topics
Foundations	1	Aug. 21	Introduction & setup
	2	Aug. 24–28	Ethics, law & responsible collection
	3	Aug. 31–Sep. 4	The post-API age
	4	Sep. 9–11	Data formats: XML & JSON
	5	Sep. 14–18	Web architecture & protocols
Documents	6	Sep. 21–25	Parsing static web pages
	7	Sep. 28–Oct. 2	Archived web pages
	8	Oct. 5–9	Dynamic web pages
	9	Oct. 12–16	PDFs
APIs	10	Oct. 19–23	Wikipedia APIs
	11	Oct. 26–30	Government data APIs
	12	Nov. 2–6	Social media APIs
	13	Nov. 9–13	AI APIs
Practice	14	Nov. 16–20	Automating data collection
	15	Nov. 23–27	**No class: Fall Break**
	16	Nov. 30–Dec. 4	Final presentations
		Final Project due Monday, December 7	

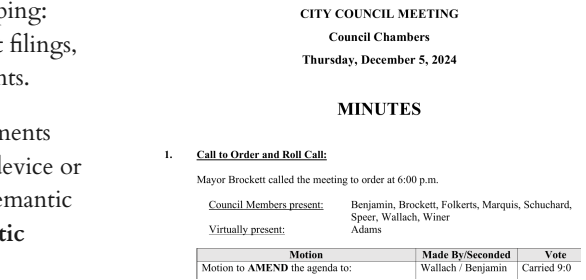
1. Monday · Concepts

The web's filing cabinets

PDFs are everywhere in institutional record-keeping: council minutes, bill texts, financial reports, court filings, academic papers, environmental impact assessments.

They are the format of choice for "official" documents because they **preserve visual layout** across any device or printer. But that priority — visual fidelity over semantic structure — makes them **hostile to programmatic extraction**.

The records inside are *readymade* data (Salganik (2018)). Somebody made them for administration, not for research. With effort you can still use them to answer questions that their authors never expected.



CITY COUNCIL MEETING

Council Chambers

Thursday, December 5, 2024

MINUTES

1. Call to Order and Roll Call:

Mayor Brockett called the meeting to order at 6:00 p.m.

Council Members present: Benjamin, Brockett, Folkerts, Marquis, Schuchard, Speer, Wallach, Winer

Virtually present: Adams

Motion	Made By/Seconded	Vote
Motion to **AMEND** the agenda to:	Wallach / Benjamin	Carried 9:0

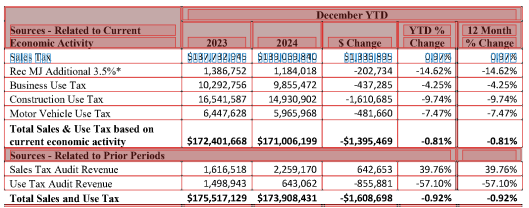
Boulder City Council minutes, December 5, 2024: the roll call on page 1.

A computer never reads a PDF

When **you** read a PDF you see paragraphs, tables, headers, and page numbers.

A **computer** reads a different thing. It sees a sequence of drawing instructions: *put this character at* (x, y). *Draw a line from here to there. Add this image.*

So text extraction is **reconstruction, not reading** — the computer must infer logical structure from spatial layout. The results will not be perfect, but they can be good enough to turn inaccessible records into analyzable datasets.

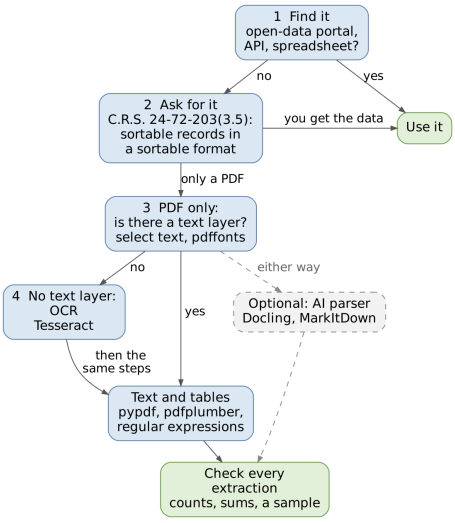


Sales and Use Tax Summary

	December YTD				
Sources - Related to Current Economic Activity	2023	2024	$ Change	YTD % Change	12 Month % Change
Sales Tax	$137,732,845	$139,089,840	$1,356,995	0.97%	0.97%
Rec MJ Additional 3.5%*	1,386,752	1,184,018	-202,734	-14.62%	-14.62%
Business Use Tax	10,292,756	9,855,472	-437,285	-4.25%	-4.25%
Construction Use Tax	16,541,587	14,930,902	-1,610,685	-9.74%	-9.74%
Motor Vehicle Use Tax	6,447,628	5,965,968	-481,660	-7.47%	-7.47%
Total Sales & Use Tax based on current economic activity	**$172,401,668**	**$171,006,199**	**-$1,395,469**	**-0.81%**	**-0.81%**
Sources - Related to Prior Periods					
Sales Tax Audit Revenue	1,616,518	2,259,170	642,653	39.76%	39.76%
Use Tax Audit Revenue	1,498,943	643,062	-855,881	-57.10%	-57.10%
Total Sales and Use Tax	**$175,517,129**	**$173,908,431**	**-$1,608,698**	**-0.92%**	**-0.92%**

Boulder's December 2024 revenue report, as pdfplumber sees it: each character placed on its own (blue boxes, two rows shown), and the table's lines are 290 thin rectangles (red).

Four steps, from easiest to hardest



Take the first step that works.

1. **Find it** in a form made for machines: an open-data portal, an API, or a spreadsheet.
2. **Ask for it.** Colorado's open-records law says a record kept in a sortable format must be provided in that format (C.R.S. 24-72-203(3.5)).
3. **Parse the PDF.** Only a PDF, and no DOCX or XLSX? Check that it is born-digital, then find its text and tables.
4. **OCR.** No text layer? Run OCR, then do step 3's work on the text it gives you.

Whatever route the data takes, **check it** before you use it. An AI parser is an optional branch at steps 3 and 4.

Today: every step by hand. Wednesday: steps 3 and 4 in Python.

Getting text and data out of a PDF

- **Text layer** — the characters a PDF stores, each with a font and a position. Select it, search it, copy it.
- **Born-digital PDF** — saved from Word, Excel, or a web page. It has a text layer.
- **Scanned (image-only) PDF** — photographs of paper pages. No text layer: nothing to select.
- **OCR**, optical character recognition — software that recognizes letters in an image and writes them out as text. **Tesseract** is the free, open-source engine most tools use.
- **Searchable PDF** — the page image with an invisible OCR text layer on top. `tesseract` makes one from a page image.
- **Layout analysis** — working out columns, reading order, headers, and footers from where text sits.
- **Table extraction** — rebuilding rows and columns from lines and positions. Tabula, `pdfplumber`, `camelot`.
- **Handwriting recognition** (HTR) — OCR for handwriting. Much less accurate than print.
- **Document AI** — a vision-language model reads the page image and writes text or a table. Fast and fluent, and it can invent what it can't read: check it against the page.

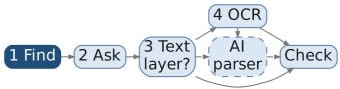
Steps 1 and 2: before you parse anything

1. Look for the data in a form made for machines.

- An **open-data portal**: a city's, a state's, or `data.gov`.
- An **API** (weeks 10–13), which returns JSON instead of pages.
- A **spreadsheet** or CSV posted beside the PDF, or the same report as `.xlsx` or `.docx`.

2. Ask for it. Colorado's open-records law says that a record kept in a sortable format, such as a spreadsheet, must be provided in that format (C.R.S. 24-72-203(3.5)).

Only a PDF? Go on to step 3.

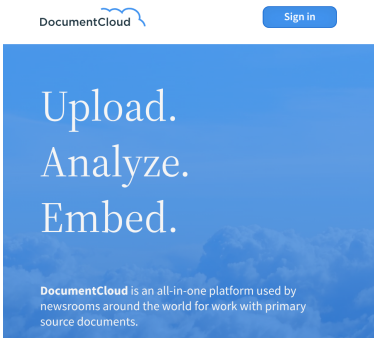


Soft enclosure & the infrastructure of transparency

Some governments publish minutes, budgets, and filings *only* as PDFs. The post-API chapter calls this **soft enclosure**. The records are public in name. In practice you cannot analyze them.

Building extraction skills enacts the **oversight** and **ownership** values — public records belong to the public, and technical barriers are themselves a form of enclosure.

Adobe created the PDF in 1993 for visual fidelity across devices — at the cost of semantic accessibility. **DocumentCloud** (ProPublica & IRE), **MuckRock** (automating FOIA), and the **Sunlight Foundation** built the infrastructure that bridges public documents and accessible data.



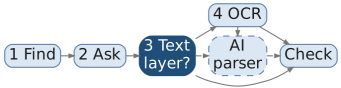
DocumentCloud: shared infrastructure for analyzing document dumps.

Step 3: is the PDF born-digital?

A born-digital PDF has a **text layer**: the characters are in the file. Check before you choose a tool:

- **Select or search.** Drag across a line, or press Ctrl+F (Cmd+F). If nothing highlights, the page is a picture.
- **Who made it?** Acrobat Reader's *File* → *Properties* names the application: Word or Excel, or a scanner.
- **Fonts.** `pdffonts report.pdf` lists none for a page that is only an image.
- **Every page.** A report can mix both: text pages, and a page that is a picture of a table.

Yes: get the text and tables out, by hand today and in Python on Wednesday. **No:** step 4.



Step 3 by hand: readers and office tools

Try this before you write code. One table, once, rarely needs Python.

- **Copy and paste** from any reader. Paragraphs come through. A table arrives as runs of text that you split into columns yourself (Excel: *Data* → *Text to Columns*).
- **Adobe Acrobat Reader** (free): *File* → *Save As Other* → *Text*. Export to Word or Excel needs a paid Acrobat subscription.
- **Microsoft Word**: *File* → *Open* a PDF, and Word converts it to an editable document. Best on PDFs that are mostly text.
- **Microsoft Excel** (Microsoft 365, Windows): *Data* → *Get Data* → *From File* → *From PDF* lists the tables it finds. Not in Excel for Mac.
- **Tabula**, a free desktop app made for journalists: draw a box around a table, and save it as CSV.

Step 3 at the command line

```
pdfinfo report.pdf # pages, producer, dates
pdffonts report.pdf # no fonts: no text layer
pdftotext -layout report.pdf report.txt
```

In a notebook, start each line with `!`, as in Chapter 5.

`pdfinfo`, `pdffonts`, and `pdftotext` come with **Poppler**: `conda install -c conda-forge poppler`.

`-layout` keeps columns lined up with spaces, which makes a table easier to split.

Step 3 online: converters, and the upload

Smallpdf, iLovePDF, Adobe's online converter, and Google Drive will all turn a PDF into Word, Excel, or text in your browser.

Each one **sends your file to a company's server**. That is fine for a city's public revenue report. It is not fine for records with people's names, health or student information, or anything you promised to keep private.

Read the size limits and the terms. Keep the original PDF, and write down which tool made which file: that is your provenance.

Whatever the tool

Check its output against the page. Every route here drops, merges, or misreads something, and none of them says so.

In class: the city's revenue report, by hand (20 minutes)

Work in pairs. Use no Python today.

1. Download Boulder's December 2024 revenue report from the textbook: `data/ch-09/revenue-report-2024-12.pdf`. (5 min)
2. Get the **Sales and Use Tax Summary** on page 2 into a spreadsheet. Use copy and paste. Then use one other tool from step 3. (8 min)
3. Clean it. Make one row per source, with the columns `source`, `2023`, and `2024`. Remove the `$` signs and the commas. (4 min)
4. Check it. Add the five sources for 2024. Compare the sum with the row "Total Sales & Use Tax based on current economic activity." (3 min)

Fast? Try the same steps on page 15, Exhibit 3. Nothing on it can be selected: that is step 4.

Report back

- Which route kept the rows and columns?
- What did you fix by hand?
- Did your sum match? If not, why not?

The file

`https://github.com/cuinfoscience/Web-Data-Science-Book/tree/main/data/ch-09`

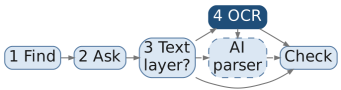
Step 4: no text layer — OCR, then the same steps

By hand. Google Drive: right-click → *Open with* → *Google Docs* runs OCR on a PDF or image of 2 MB or less. Tables and columns are usually lost.

At the command line, one page at a time:

```
pdftoppm -png -r 300 -f 15 -l 15 report.pdf page
tesseract page-15.png page-15 # text
tesseract page-15.png page-15 pdf # searchable PDF
```

Then do step 3's work on the text OCR gives you. OCR misreads characters (a period for a comma, "l" for 1), so the checks matter more here. An **AI parser** is an optional branch at steps 3 and 4; Wednesday shows one.



`tesseract` and `pytesseract` are in `webdata`. OCRmyPDF, for a whole PDF, is optional: Chapter 9's appendix.

In the notebook

This week's companion notebook follows the same steps:

- **Setup —** install pypdf, pdfplumber, pytesseract, and Tesseract, once
- **Step 3: is there a text layer? —** characters and images on every page
- **Step 3: text and tables —** pypdf for a page's text, pdfplumber for words by position and tables rebuilt from rules
- **Step 4: OCR —** text from a scan, or from a picture of a table
- **Check —** counts, sums, two tools, two documents, and a sample coded by hand
- **Case studies —** attendance in council minutes with pypdf, and revenue reports with pdfplumber
- **Optional branch —** AI parsers such as Docling, judged with the same checks

2. Wednesday · Notebook Lab

Daily note questions

- **Are these tools better than AI at extracting information from PDFs? —** Definitely more painful, AI document understanding frameworks are what I would recommend
- **Why are PDFs standard? —** Open standard, adoption, path dependence
- **How to read figures and charts? —** OCR and extract text or use AI DU frameworks
- **Copying and pasting table into Excel —** Use the Text-to-Columns button under Data tab

Before the first cell: install once

```
%conda install -y -c conda-forge pypdf pdfplumber pytesseract
      tesseract

import pypdf, pdfplumber, pytesseract
print(pypdf.__version__, pdfplumber.__version__)
print(pytesseract.get_tesseract_version())
# 6.20.1 0.11.10
# 5.5.3
```

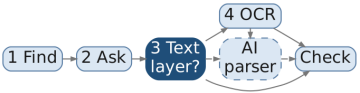
A `webdata` made from today's setup page has all four. An older one, or Anaconda's `base`, needs the first cell; then restart the kernel.

`pytesseract` is the Python library; `tesseract` is the OCR program it runs, and `pip` can't install it.

Step 3 in code: is there a text layer?

```
def page_profile(path):
    reader = PdfReader(path)
    characters = []
    images = []
    for page in reader.pages:
        characters.append(len(page.extract_text()))
        images.append(len(page.images))
    return characters, images

characters, images = page_profile(report_2024)
print(characters) # [1332, 2279, ..., 4624, 265]
print(images) # [1, 0, ..., 0, 1]
```



Text on every page of the December 2024 report: born-digital. But the last page has one image and 265 characters, the note beside a **picture** of a table: that page is step 4.

One image per page and plenty of text, as in the 2018 minutes, is a **scan with a text layer**: test that text before you trust it.

pypdf: text in the order it was drawn

```
from pypdf import PdfReader

minutes_2024 = PdfReader(get_saved_copy("council-minutes-2024-12-05.pdf"))
text = minutes_2024.pages[0].extract_text()
# P age 1
# CI
# TY COUNCIL MEETING
# ...
# M
# ayor Brockett called the meeting to order at 6:00 p.m.
```

Every word is there, and some are broken. Word drew the “M” of “Mayor”, jumped to another line, and came back: pypdf starts a new line at each jump. `clean_page_text()` in the notebook removes “P age 1” and rejoins a first letter to the rest of its word.

The one line that flattens the page

Tempting one-liner:

```
# DON'T: \s matches newlines too,
# so this flattens the whole page
re.sub(r'\s+', ' ', raw_text)
```

Correct — collapse spaces & tabs, **keep** the line breaks:

```
text = re.sub(r'[ \t]+', ' ', raw_text)
kept = []
for line in text.split('\n'):
    if line.strip(): # skip empty lines
        kept.append(line.strip())
clean = '\n'.join(kept)
```

The most important choice is what *not* to collapse.

Line structure is real information: a roll call's names, and each heading, sit on lines of their own.

Erase the newlines and no pattern can find where a list starts or ends.

pdfplumber: words where they sit

```
import pdfplumber

with pdfplumber.open(report_2024) as report:
    page = report.pages[1]
    tables = page.find_tables()
    rows = tables[0].extract()
print(rows[-1][:2])
# ['Use Tax Audit Revenue', None]
```

pdfplumber groups characters by **position**, so "Mayor" comes out whole. Its table finder rebuilds a grid from the page's rules. On the December 2024 summary page it stops one row short: the **total** has no vertical rules beside it, and nothing raises an error.

Sales and Use Tax Summary

	December YTD				
Sources - Related to Current Economic Activity	**2023**	**2024**	**$ Change**	**YTD % Change**	**12 Month % Change**
Sales Tax	$137,732,945	$139,069,840	$1,336,895	0.97%	0.97%
Rec MJ Additional 3.5%*	1,386,752	1,184,018	-202,734	-14.62%	-14.62%
Business Use Tax	10,292,756	9,855,472	-437,285	-4.25%	-4.25%
Construction Use Tax	16,541,587	14,930,902	-1,610,685	-9.74%	-9.74%
Motor Vehicle Use Tax	6,447,628	5,965,968	-481,660	-7.47%	-7.47%
Total Sales & Use Tax based on current economic activity	**$172,401,668**	**$171,006,199**	**-$1,395,469**	**-0.81%**	**-0.81%**
Sources - Related to Prior Periods					
Sales Tax Audit Revenue	1,616,518	2,259,170	642,653	39.76%	39.76%
Use Tax Audit Revenue	1,498,943	643,062	-855,881	-57.10%	-57.10%
Total Sales and Use Tax	**$175,517,129**	**$173,908,431**	**-$1,608,698**	**-0.92%**	**-0.92%**

pdfplumber's table finder on the summary page: shaded where it found cells.

The total, at the bottom, is left out.

pdfplumber's table settings

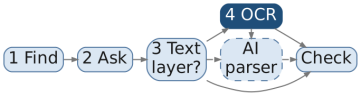
`find_tables()` and `extract_table()` take one dictionary of settings:

- **Strategies**: `"lines"` (rules and rectangle sides, the default), `"lines_strict"`, `"text"` (words that line up), `"explicit"` (positions you give).
- `explicit_vertical_lines`: the column edges yourself, as the chapter does to get the totals row back.
- **Tolerances**, 3 points each: `snap` (merge near-parallel rules), `join` (join broken rules), `intersection` (where rules cross), `edge_min_length` (drop short marks).
- **Text strategy**: `min_words_vertical` (3), `text_x_tolerance`.
- **See it**: `page.to_image().debug_tablefinder(settings)`.

All of them, with defaults: `https://github.com/jsvine/pdfplumber#table-extraction-settings`

Step 4 in Python: OCR, then the same steps

1. **Get the image.** pypdf hands you a page's picture: `page.images[0].image`.
2. **OCR it.** `pytesseract.image_to_string()` runs Tesseract and returns text.
3. **Same steps as step 3.** Split the text into lines, and read each row with a regular expression.
4. **Check harder.** Set aside the lines that don't parse, and add up each row: OCR's misread digits still look like numbers.

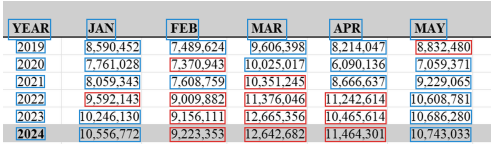


No text layer: OCR

```
import pytesseract

exhibit_3 = (PdfReader(report_2024)
    .pages[14].images[0].image.convert("L"))
ocr_text = pytesseract.image_to_string(
    exhibit_3, config="--psm 6")
# 2021 8,059,343 7,608.759 10,351,245 ...
```

Exhibit 3 of the December 2024 report is a **picture** of a table. Tesseract reads it, and puts periods for commas in a few figures. The rows' sums catch every misread digit; Tesseract's confidence scores don't.



YEAR	JAN	FEB	MAR	APR	MAY
2019	8,590,452	7,489,624	9,606,398	8,214,047	8,832,480
2020	7,761,028	7,370.943	10,025,017	6,090,136	7,059,371
2021	8,059,343	7,608,759	10,351,245	8,666,637	9,229,065
2022	9,592,143	9,009,882	11,376,046	11,242,614	10,608,781
2023	10,246,130	9,156,111	12,665,356	10,465,614	10,686,280
2024	10,556,772	9,223,353	12,642,682	11,464,301	10,743,033

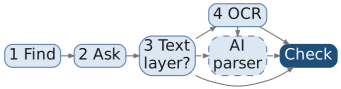
Each word Tesseract read: blue if its confidence is 80 or more, red if less.

Most red words are right.

Check every extraction

Extraction fails without errors. Each of these caught a real problem in Chapter 9:

1. **Look at the page beside the output.**
2. **Count.** The summary table has nine rows of figures; the council has nine seats.
3. **Sums.** The five sources add up to $171,006,200; the printed subtotal is $171,006,199.
4. **Two tools agree.** pypdf's text and pdfplumber's table give the same 2024 figures.
5. **Two documents agree.** A year printed in two reports should match.
6. **A sample coded by hand.** Read some pages yourself, and measure the extraction against them.



The case studies, on the pathway

Both start at step 3: the chapter works from PDFs saved from the city's records portal.

- **Council minutes, 2018–2024** (case study 1). From 2020, born-digital: step 3, with pypdf. 2018 and 2019 are **scans with a text layer**: step 3, but test that text first — the scanner's OCR read "COUNCIL" as "COTJNCIL".
- **Revenue reports** (case study 2). Born-digital, so step 3, with pdfplumber: read the totals row by position.
- **Exhibit 3**, the picture of a table on the last page of the December 2024 report: step 4, as on the OCR slide.

Each case study ends at the last box: the attendance counts match a hand count, and the totals match the reports' own change columns.

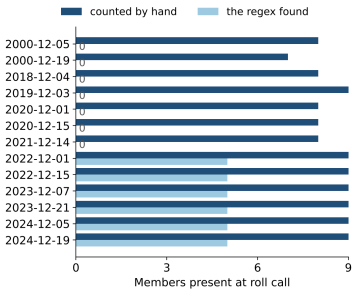
Case study 1: the obvious regex

```
present = re.search(r'Present:\s*(.*?)\s*(?:Absent:|$)',
                    page_1, re.IGNORECASE | re.MULTILINE)
```

Eleven December meetings, 2018–2024, read with pypdf. This pattern finds **nobody** in the five meetings before 2022, and **5 of 9** members after.

The minutes recorded the roll call two ways:

- through 2020, a sentence: “Council Members Brockett, … and Young were present.” The mayor isn’t in it.
- from 2021, a label and a list that wraps onto a second line, with a separate label for members online.



Members present at roll call, with the two December 2000 scans: counted by hand (dark), and found by this regex (light).

Case study 1: one function, both formats

```
def roll_call(page_1):
    present = set()
    mayor = re.search(r"Mayor (\w+) called the meeting to order", page_1)
    if mayor:
        present.add(mayor.group(1))
    # 2018-2020: "Council Members Brockett, ... and Young were present"
    sentence = re.search(r"Council Members ([A-Z][^:]*?)\s+were\s+present", page_1)
    if sentence:
        present |= split_names(sentence.group(1))
    # 2021-2024: "Council Members present:", "Virtually present:", "Present virtually:"
    ... # each list, plus the next line while the list ends with a comma
    return present
```

It matches a hand count of all eleven meetings: 8 or 9 members at each roll call. A member who arrived late counts as absent, and one member is “Swetlik” in 2019 and “Swetlick” in 2020.

Case study 2: read the totals row by position

```
def row_words(page, label):
    found = page.search(label)[0] # where the label sits, in points
    # a strip of the page, as tall as the label's line
    line = page.crop((0, found["top"] - 1, page.width, found["bottom"] + 1))
    words = []
    for word in line.extract_words():
        if word["x0"] > found["x1"]: # right of the label
            words.append(word["text"])
    return words

row_words(page, "Total Sales and Use Tax") # the 2018 report
# ['$134,538,892', '$140,648,710', '$6,109,818', '4.54%']
```

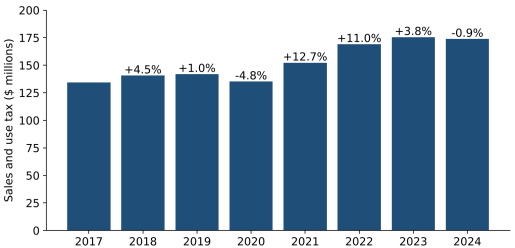
Seven year-end revenue reports. Find the summary page by its text: 2020's report opens with a letter restating earlier figures, so its summary is on page 3. Then skip the table finder, and read the one row you need by where it sits. Each report's own change column checks the two totals.

Case study 2: one series from seven reports

Six years are printed twice: in their own report, and as “the year before” in the next. Five agree to the dollar.

2018 doesn’t. The 2019 report gives it as $38,959 less than the 2018 report did, and doesn’t say why. Use each year’s latest printing, and say so.

Sales and use tax fell **4.76%** in 2020, then rose **12.65%** and **11.04%**. In dollars of each year: not adjusted for inflation.

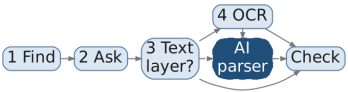


Total sales and use tax by year, read with pdfplumber, with the change from the year before.

The optional branch: AI parsers

- **Text-layer converters**: MarkItDown. Fast; a scan gives it nothing.
- **Pipelines**: layout + table + OCR models. Docling runs on a laptop.
- **Vision-language models**: olmOCR, DeepSeek-OCR, Datalab's Chandra. Most need a GPU, or a hosted service that receives your file.
- **Pandoc** converts the Markdown they write to HTML or Word. It can't read a PDF.

None of these is in `webdata` or needed for the lab: Chapter 9, "Document Understanding."



Docling on Exhibit 3, the picture of a table:

- 21 of 30 rows whole, and each adds up
- 9 incomplete or merged; 2 under the wrong label
- no error raised

Same checks as ever: count, sums, labels, two tools.

Lab: work these, then show-and-tell

Work in pairs. Run both case studies in the notebook, then extend one:

1. Run `roll_call()` on the two December 2000 minutes. They are scans. What does the pattern miss? Fix it, and check against the page.
2. Count the votes in each meeting with the tally pattern (`Carried 9:0`). What share were unanimous?
3. Two members arrived after the roll call, in December 2018 and 2021. Find what the minutes say, and decide how to count them.
4. Read the **Sales Tax** row from all seven reports with `row_words()`. Plot it beside the total.
5. Read the 2023 summary table with `find_tables()`. Which rows does it find? Compare with `row_words()`.
6. **5617**: write a memo about the two datasets you built. Use the categories of Gebru et al. (2021)

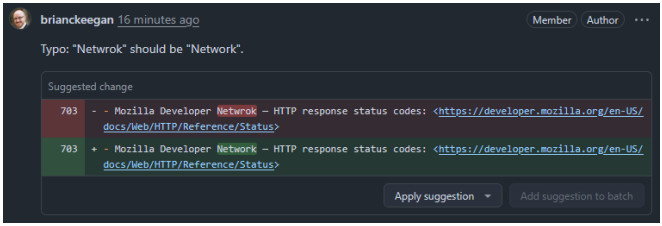
Show-and-Tell

Bring one of these to class:

- a bug that you could not solve
- data that you collected and find interesting
- a question for a research design

3. Friday · Textbook Revisions

Improving Chapter 9 together



Today we make the book better. Each of you opens a **pull request** against `ch-09-pdfs.qmd` and reviews a classmate's.

The workflow (from Week 1):

1. Branch / fork the book repo
2. Edit the chapter; commit with a clear message
3. Open a PR describing *what* and *why*
4. Review two peers' PRs: kind, specific, actionable

A revision menu for Chapter 9

High-value targets — pick one and scope it to a single PR:

- **Run it against real Boulder minutes.** The `# Returns:` outputs and five regex patterns are *illustrative*. Download actual signed minutes, report what the patterns catch and miss, and label sample output as representative.
- **Add the chapter's first figures.** It is all code and prose today: a "drawing-instructions vs. logical text" diagram, a pipeline flowchart, or an annotated minutes screenshot would anchor Monday's core idea.
- **Harden `extract_attendance`.** The chapter itself flags "Absent: None," title variants ("Council Member Smith"), and semicolon / line-break separators. Contribute handling **plus tests** for the formats you actually hit.
- **Make an exercise self-checking.** Add expected output, a rubric, or a starter cell to the open-ended "find and extract" and "topic tracking" exercises.
- **Deepen "When PyPDF Is Not Enough."** Turn the two-line `pdfplumber` / OCR sketches into a runnable table-extraction or Tesseract example tied to "Further Reading."

A good revision, and a good review

A strong PR

- One focused change, clearly titled
- Explains the problem it fixes
- Builds (`quarto render`) without errors
- Matches the book's voice and notation
- Discloses any AI assistance

A strong review

- Runs / reads the change, not just skims
- Names specifics (line, claim, code)
- Separates "must fix" from "nice to have"
- Is generous — assume good faith
- Ends with a clear verdict

Next week: a new module on **APIs** — when data arrives as JSON and no parsing is required (starting with Wikipedia).

Key takeaways

1. **Take the first step that works**: find the data in a form made for machines, ask for it, parse a born-digital PDF, and only then OCR.
2. PDFs optimize **visual fidelity over structure** — extraction is reconstruction, not reading.
3. `pypdf` gives text in the order it was drawn; expect breaks inside words and plan to clean them.
4. Collapse spaces/tabs but **keep newlines** — line structure is information; regex imposes the rest.
5. `pdfplumber` keeps each word's position: find a label, then read its row. OCR is for pages with no text layer.
6. **Check every extraction** against a count by hand or a total the document prints — across all its formats.